Chapter 3 – Operator Overloading

(*Object-Oriented Programming*)

Outline

3.1 Introduction

3.2 Restrictions On Operator Overloading

3.3 Operator Function

3.4 Implementation of Operator Function

3.1 Introduction

```
#pragma once

class Fraction
{
private:
    int numerator;
    int denominator;

public:
    Fraction(int num = 0, int denom = 1);
    Fraction add(const Fraction &f);
    int getNum() const { return numerator; }
    int getDenom() const { return denominator;}
    ~Fraction(void);
};
```

```
int main()
{
    Fraction f1(3,2), f2(1,6);
    f1 = f1.add(f2);
    cout << f1.getNum() << "/" << f1.getDenom() << endl;
    return 0;
}
```

Why is operator overloading necessary?

- To add fractions, we must call a method:

```
f1 = f1.add(f2);
```

- What if we want to add multiple fractions?

```
f1 = f1.add(f2).add(f3);
```

→Code becomes **less intuitive and harder to read.** Can we write it like this using natural mathematical syntax?

```
f1 = f1 + f2 + f3;
```

More readable, more intuitive, similar to built-in data types.

Why is operator overloading necessary?

- To display a fraction, we must call methods:

```
cout << f1.getNum() << "/" << f1.getDenom();
```

- How to display multiple fractions?

```
cout << f1.getNum() << "/" << f1.getDenom() << endl;
cout << f2.getNum() << "/" << f2.getDenom() << endl;
```

- Can we write it like this?

```
cout << f1 << endl;
cout << f2 << endl;
```

Why is operator overloading necessary?

- **Problem:** The following statements are **not valid**:

```
f1 = f1 + f2;
cout << f1 << endl;
```

→ Because operators like + and << are only defined for built-in data types: int, long float, double, char, …

- **Solution:** C++ provides a powerful feature "Operator Overloading".
  - Allows creating new versions of existing operators.
  - Enables operators to work with user-defined types (classes).
  - Allow objects to use operators like built-in types.
  - The original meaning of operators is preserved.
  - Operators can behave differently depending on the context and operand types.

Using an Operator on Objects

- To use an operator on an object of a class, you must define overloaded operator functions for that class—with three exceptions:
  - assignment operator (=): perform memberwise assignment of the data members.
  - address (&) operator: returns a pointer to the object.
  - comma operator: evaluates the expression to its left then the expression to its right, and returns the value of the latter expression.

Proper Use of Operator Overloading

- Proper use of operator overloading improves usability for class users.
- However, it should not be overused.
  - Only overload operators when they are necessary and meaningful.
  - The meaning must be appropriate to the context.
- Example: Suppose we build a class MyList to store a list of elements.

```
MyList a, b;
a + b;   // used to insert elements?
a - b;   // used to remove elements?
```

→ This can be **misleading and confusing,** lead to **incorrect usage or errors.**

→ Better approach:

```
a.insert(x);
a.remove(x);
```

3.2 Restrictions On Operator Overloading

- The precedence of an operator cannot be changed.
- The associativity of an operator cannot be changed.
- Cannot change the "arity" of an operator (the number of operands an operator takes)
  - Overloaded unary operators remain unary operators;
  - Overloaded binary operators remain binary operators;
  - Operators &, *, + and - all have both unary and binary versions; these unary and binary versions can be separately overloaded.
- Cannot create new operators.
- The meaning of how an operator works on values of fundamental types cannot be changed.
- Related operators, like + and +=, must be overloaded separately.
- No overloading operators for built-in types.

Restrictions On Operator Overloading

Operators that can be overloaded							
+	-	*	/	%	^	&	|
~	!	=	<	>	+=	-=	*=
/=	%=	^=	&=	|=	<<	>>	>>=
<<=	==	!=	<=	>=	&&	||	++
--	->*	,	->	[]	()	new	delete
new[]	delete[]						

Operators that cannot be overloaded				
.	.*	::	?:	sizeof

3.3 Operator Function

- Operators can be redefined for user-defined types.
  - Using a special function called an operator function.
  - Operator overloading is a form of function overloading.
- Operator functions are functions with special function names.
- General syntax:

```
return_type   operator<@>(parameter_list)
{
        //body
}
```

Here, @ represents an operator symbol in C++ (such as +, -, *, =, etc.).

Example

```
class Fraction
{
private:
    int numerator;
    int denominator;
public:
    Fraction( int num, int denom );
    //Fraction add(const Fraction &f);
    Fraction operator+( const Fraction& other ) const;
    Fraction operator+( int val ) const;
    Fraction& operator+=( const Fraction& other );
    Fraction& operator+=( int val );

    int getNum() const { return numerator; }
    int getDenom() const { return denominator; }
    ~Fraction(void);
};
```

Example

```
Fraction Fraction::operator+( const Fraction& other ) const
{
    // Compute GCD of denominators using Euclid's algorithm
    int a = denominator, b = other.denominator;
    while( b != 0 ) {
        int t = b;
        b = a % b;
        a = t;
    }
    // Compute LCM from GCD
    int lcm = denominator / a * other.denominator;
    // Return a new fraction
    return Fraction(
        (lcm / denominator) * numerator +
        (lcm / other.denominator) * other.numerator, lcm );
}
```

Example

```
Fraction Fraction::operator+( int val ) const
{
    // Convert int to fraction then reuse operator+(Fraction)
    return *this + Fraction( val, 1 );
}

Fraction& Fraction::operator+=( const Fraction& other ) {
    // Use the already defined "+" operator
    *this = *this + other;
    // Return a reference to ourself
    return *this;
}

Fraction& Fraction::operator+=( int val ) {
    *this = *this + val;
    return *this;
}
```

Calling Operator Functions

Every overloaded operator can be called in two equivalent ways:

```
int main()
{

    Fraction f1(1, 2), f2(1, 3), f3;

    // Using operator syntax (recommended)
    f3 = f1 + f2; //operator+(const Fraction& other)
    f3 = f1 + 2; //operator+(int val)

    // Using explicit function call (equivalent)
    f3 = f1.operator+(f2);
    f3 = f1.operator+(2);

    return 0;
}
```

Calling Operator Functions

Note: commutativity is NOT automatic

```
f3 = f1 + 2;     // OK
f3 = 2 + f1;    //?
```

Can we write `2 + f1` ? If yes, which function gets called? If no, why not?

3.4 Implementation of Operator Function

- An operator function can be implemented in **two ways:**
  - As a Member Function
  - As a Non-member Function

Number of Parameters

- When the operator function is a member function:
  - The number of parameters is reduced by 1, because the calling object (*this) serves as the implicit first operand.
- When the operator function is a non-member function:
  - All operands must be passed as explicit parameters — no implicit argument.

	Unary	Binary
Member *	0	1 **
Non-member	1	2

* Except: =, [], (), -> — must always be member functions.

** Except: postfix ++/-- — takes a dummy int parameter.

Operator Function As a Member Function

- The operator function is defined inside the class.
- Must be non-static — a left-hand object is required to invoke it.
- When to use for class A:
  - The left operand is always an object of the class A.
  - The left operand is modified after the operator executes (e.g., +=, ++, --).
  - The function needs direct access to private data members.
  - Operators that must be member functions: =, [], (), ->.

```
Fraction operator+( const Fraction& other )
{
        // *this  = left operand
        // other  = right operand
}
```

Operator Function As a Non–Member Function

- The operator function is defined outside the class.
  - Global function: use public getters/setters.
  - Friend function: if needs to access private or protected members.
- When to use for class A:
  - The left operand is NOT an object of class A.
  - The left operand is NOT modified after the operator executes.
  - Type conversion on the left operand must be allowed.
  - Enable the operator to be commutative.

```
Fraction f1(0,1);
cin >> f1; //The left operand is istream
f1 = 2 + f1; //The left operand is int
cout << f1 << endl; //The left operand is ostream
```

Guidelines for Implementing Operator Functions

Part 1 — Arithmetic & Assignment				
Operator	**Name or Category**	**Method or Global Friend**	**When to Overload**	**Sample Prototype**

`operator+`
`operator-`
`operator*`
`operator/`
`operator%`
	Binary arithmetic	Global friend recommended	Provide arithmetic operations for class	`friend const T operator+(const T&, const T&);`

`operator-`
`operator+`
`operator~`
	Unary arithmetic & bitwise	Method recommended	Provide unary operations for class	`const T operator-() const;`

`operator++` `operator--`	Increment & decrement	Method recommended	When overloading binary + and -	`T& operator++();` `const T operator++(int);`
`operator=`	Assignment	Method required	Object has dynamically allocated memory or to prevent assignment	`T& operator=(const T&);`

`operator+=`
`operator-=`
`operator*=`
`operator/=`
`operator%=`
	Shorthand arithmetic assignments	Method recommended	When overloading binary arithmetic operators	`T& operator+=(const T&);`

`operator==` `operator!=`	Equality	Global friend recommended	When objects need to be compared	`friend bool operator==(const T&, const T&);`

Guidelines for Implementing Operator Functions

Part 2 — Comparison, Stream & Access				
Operator	**Name or Category**	**Method or Global Friend**	**When to Overload**	**Sample Prototype**

`operator<`
`operator>`
`operator<=`
`operator>=`
	Relational	Global friend recommended	When objects need ordering/sorting	`friend bool operator<(const T&, const T&);`

`operator<<` `operator>>`	Stream I/O	Global friend required	To support cout << and cin >>	`friend ostream& operator<<( ostream&, const T&);`
`operator[]`	Subscript	Method required	Provide array-style access to elements	`T& operator[](int index);` `const T& operator[](int) const;`
`operator()`	Function call	Method required	Make object callable like a function	`return_type operator()(params);`
`operator->`	Member access via pointer	Method required	Smart pointer or proxy class	`T* operator->();`

`operator&`
`operator|`
`operator^`
`operator<<`
`operator>>`
	Binary bitwise	Global friend recommended	Bitwise operations on user-defined types	`friend const T operator&(const T&, const T&);`

Guidelines for Implementing Operator Functions

Part 3 — Bitwise, Logical & Others				
Operator	**Name or Category**	**Method or Global Friend**	**When to Overload**	**Sample Prototype**

`operator&=`
`operator|=`
`operator^=`
`operator<<=`
`operator>>=`
	Bitwise assignments	Method recommended	When overloading binary bitwise operators	`T& operator&=(const T&);`

`operator!`	Logical NOT	Method recommended	Boolean-like classes	`bool operator!() const;`
`operator&&` `operator||`	Logical AND/OR	Global friend (use with caution)	Rarely needed; avoid if possible	`friend bool operator&&(const T&, const T&);`
`operator,`	Comma	Global friend (rarely used)	Rarely overloaded; avoid in most cases	`friend T operator,(const T&, const T&);`

`operator new`
`operator delete`
`operator new[]`
`operator delete[]`
	Memory management	Method or global	Custom memory allocation strategy	`void* operator new(size_t size);`
				`void operator delete(void* ptr);`

`operator T` (conversion)	Type conversion	Method required	Implicit/explicit conversion to type T	`operator T() const;`

Example: Overloading << And >> Operators

- Overload these operators to perform input and output for user-defined types.

```
#include <iostream>
#include <string>
using namespace std;

class Student
{
private:
    string id;
    string name;
public:
    friend ostream& operator <<(ostream&, const Student&);
    friend istream& operator >>(istream&, Student&);
};
```

Example: Overloading << And >> Operators

```
ostream& operator <<(ostream& out, const Student& s)
{
     out << "ID is " << s.id << endl;
     out << "Name is " << s.name << endl;
     return out;
}

istream& operator >>(istream& in, Student& s)
{
     cout << "Enter ID: ";
     in >> s.id;
     in.ignore();
     cout << "Enter the name: ";
     getline(in,s.name);
     return in;
}
```

```
int main()
{
     Student s;
     cin >> s;
     cout << "Information of the student:\n" << s << endl;
    return 0;
}
```

Example: Overloading Unary Operators

```
#include<iostream>
using namespace std;

class Vector2D {
private:
    double x, y;
public:
    Vector2D( double x = 0, double y = 0) : x(x), y(y) {}

    // As a Member Function: *this is the implicit operand
    Vector2D operator-() const { return Vector2D( -x, -y ); }
};

int main()
{
    Vector2D v1(3,4);
    Vector2D v2 = -v1;  //v1.operator-()
}
```

Example: Overloading Unary Operators

```
#include<iostream>
using namespace std;

class Vector2D {
private:
    double x, y;
public:
    Vector2D( double x, double y ) : x(x), y(y) {}
    friend Vector2D operator-( const Vector2D& v );  // declare friend
};

// As a Non-member Function: operand passed explicitly
Vector2D operator-( const Vector2D& v ) {
    return Vector2D( -v.x, -v.y );
}

int main() {
    Vector2D v1(3,4);
    Vector2D v2 = -v1;     //operator-(v1)
}
```

Example: Overloading Binary Operators

```
#include<iostream>
using namespace std;

class Student {
private:
    string name;
    double gpa;
public:
    Student( string name, double gpa ) : name(name), gpa(gpa) {}
    friend bool operator<(const Student& a, const Student& b);
    friend bool operator>(const Student& a, const Student& b);
};
```

Example: Overloading Binary Operators

```
// Less than: lower gpa
bool operator<( const Student& a, const Student& b ) {
    return a.gpa < b.gpa;
}

// Greater than: greater gpa
bool operator>( const Student& a, const Student& b ) {
    return b < a; //reuses operator <
}

int main()
{
    Student s1("Lan", 7.5);
    Student s2("Hung", 8);
    cout << ( s2 < s1 ) <<endl;  // operator<(s2,s1)
    cout << ( s1 > s2 ) <<endl;  // operator>(s1,s2)
}
```

Example: Overloading Binary Operators

```
#include<iostream>
using namespace std;
class Student {
private:
    string name;
    double gpa;
public:
    Student( string name, double gpa ) : name(name), gpa(gpa) {}

    // As a Member Function: left operand is *this

    bool operator<(const Student& other) const {
        return gpa < other.gpa;
    }
    bool operator>(const Student& other) const {
        return other < *this; // reuse operator <
    }
};
```

Example: Overloading Binary Operators

```
int main()
{
    Student s1("Lan", 7.5);
    Student s2("Hung", 8);
    cout << ( s2 < s1 ) <<endl;  // s2.operator<(s1)
    cout << ( s1 > s2 ) <<endl;  // s1.operator>(s2)
}
```

Example: Overloading Conversion Operators

- For user-defined types, you must specify how conversions work — using:

1. Conversion constructor:
   - A constructor callable with one argument acts as a conversion constructor.
   - Converts a value of another type into a class object **implicitly**.
2. Conversion operator (cast operator):
   - Must be a non-static member function.
   - Return type is implicit (the target type).

*However, both cases perform implicit conversion → Unwanted Conversion

→ Use the explicit keyword to enforce explicit conversion.

Example: Overloading Conversion Operators

```
#include<iostream>
using namespace std;

class Fraction {
private:
    int numerator;
    int denominator;
public:
    // (1) Conversion constructor: int → Fraction
    Fraction( int num, int denom = 1 )
        : numerator(num), denominator(denom) {}

    // (2) Conversion operator: Fraction → double
    operator double() const {
        return (double)numerator / denominator;
    }
    int getNum() const { return numerator; }
    int getDenom() const { return denominator; }
};
```

Example: Overloading Conversion Operators

```
void printFraction( const Fraction& f )
{
    cout<<f.getNum()<<"/"<<f.getDenom()<<endl;
}

int main()
{
    Fraction f(1, 2);
    printFraction(f);    // normal call
    printFraction(3);   //int 3 → Fraction(3, 1) implicitly

    double d = f; // Fraction → double, compiler calls f.operator double() implicitly
    cout<<d<<endl;
   return 0;
}
```

Example: Overloading Conversion Operators

```
#include<iostream>
using namespace std;

class Fraction {
private:
    int numerator;
    int denominator;
public:
    // (1) Conversion constructor: int → Fraction
    explicit Fraction( int num, int denom = 1 )
        : numerator(num), denominator(denom) {}

    // (2) Conversion operator: Fraction → double
    explicit operator double() const {
        return (double)numerator / denominator;
    }
    int getNum() const { return numerator; }
    int getDenom() const { return denominator; }
};
```

Example: Overloading Conversion Operators

```
int main()
{
    Fraction f(1, 2);
    printFraction(f);    // normal call
    printFraction(Fraction(3));   //explicit constructor call

    double d = (double)f; // explicit cast required
    cout << d << endl;
   return 0;
}
```

Exercises

Build a Date Class

- Data members:
  - day — integer from 1 to 31 depending on the month.
  - month — integer from 1 to 12
  - year — integer (year >= 1900)
- Member functions:
  - Constructor with 3 parameters, default value: 1/1/1900
  - Getters / Setters
  - isValidDate() — validates whether the date is valid
- Overloading the following operators:
  - Comparison: >, <, ==
  - Stream I/O: <<, >>
  - Day increment: ++ (returns the next day)